

算法分析第3次作业

算法分析题3

3-1

3-1 设计一个 $O(n^2)$ 时间的算法，找出由 n 个数组成的序列的最长单调递增子序列。

第3-1题答案:

1. 算法设计思想

本题采用**动态规划**算法进行求解。其核心思想是将原问题分解为若干个相互重叠的子问题，通过存储子问题的解来避免重复计算，最终推导出全局最优解。

- **状态定义**：定义数组 $b[0 : n - 1]$ ，其中 $b[i]$ 表示以序列中第 i 个元素 $a[i]$ 结尾的最长单调递增子序列的长度。
- **最优子结构**：为了计算 $b[i]$ ，需要检查在 $a[i]$ 之前的所有元素 $a[k]$ （其中 $0 \leq k < i$ ）。如果满足 $a[k] \leq a[i]$ ，说明 $a[i]$ 可以接在以 $a[k]$ 结尾的递增子序列后面，形成一个更长的递增子序列。此时，该序列长度为 $b[k] + 1$ 。
- **状态转移方程**：

基于上述逻辑，可以得到递推公式：

$$b[0] = 1$$

$$b[i] = \max_{0 \leq k < i, a[k] \leq a[i]} \{b[k]\} + 1$$

若在 $a[i]$ 之前找不到任何小于等于它的元素，则 $b[i] = 1$ 。

- **最终结果**：整个序列 a 的最长单调递增子序列的长度即为数组 b 中的最大值，即 $\max_{0 \leq i < n} \{b[i]\}$ 。

2. 算法分析

- **时间复杂度**：

算法采用了双重循环结构。外层循环遍历序列中的每一个元素 $a[i]$ ，次数为 n ；内层循环针对每个 i ，遍历其之前的 i 个子问题以寻找最大长度。

计算总次数为： $\sum_{i=1}^{n-1} i = \frac{n(n-1)}{2}$ ，因此时间复杂度为 $O(n^2)$ 。

最后通过一次遍历 b 数组即可得到结果，复杂度为 $O(n)$ ，不改变整体复杂度等级。

- **空间复杂度**：

算法使用了一个长度为 n 的辅助数组 b 来存储中间状态，因此空间复杂度为 $O(n)$ 。

3-4 给定 n 种物品和一背包。物品 i 的重量是 w_i ，体积是 b_i ，其价值为 v_i ，背包的容量为 c ，容积为 d 。问应如何选择装入背包中的物品，使得装入背包中物品的总价值最大？在选择装入背包的物品时，对每种物品 i 只有两种选择，即装入背包或不装入背包。不能将物品 i 装入背包多次，也不能只装入部分的物品 i 。试设计一个解此问题的动态规划算法，并分析算法的计算复杂性。

第3-4题答案:

1. 算法设计思想

本题属于**二维 0-1 背包问题**。与经典的 0-1 背包问题相比，该问题增加了一个约束维度（体积 b_i 及其上限 d ）。由于每种物品只能选择装入一次或不装入，且要求总价值最大，该问题具有明显的**最优子结构**性质。

- **状态定义**：定义 $m(i, j, k)$ 为：当可选择物品范围为 $\{i, i+1, \dots, n\}$ ，且当前背包剩余载重上限为 j 、剩余容积上限为 k 时，能够获得的最大总价值。
- **状态转移分析**：对于第 i 种物品，存在两种选择策略：
 - 1. 不装入第 i 种物品**：此时问题的最优值等于剩余物品在相同约束下的最优值，即 $m(i+1, j, k)$ 。
 - 2. 装入第 i 种物品**：前提是当前载重 $j \geq w_i$ 且容积 $k \geq b_i$ 。装入后，背包获得价值 v_i ，剩余约束变为 $j - w_i$ 和 $k - b_i$ 。此时价值为 $v_i + m(i+1, j - w_i, k - b_i)$ 。
- **状态转移方程**：

根据上述策略，可以得到如下递归定义：

$$m(i, j, k) = \begin{cases} \max\{m(i+1, j, k), m(i+1, j - w_i, k - b_i) + v_i\} & j \geq w_i \text{ 且 } k \geq b_i \\ m(i+1, j, k) & 0 \leq j < w_i \text{ 或 } 0 \leq k < b_i \end{cases}$$

- **边界条件**：对于最后一种物品 n ：

$$m(n, j, k) = \begin{cases} v_n & j \geq w_n \text{ 且 } k \geq b_n \\ 0 & 0 \leq j < w_n \text{ 或 } 0 \leq k < b_n \end{cases}$$

- **求解目标**：通过自底向上的动态规划计算，最终 $m(1, c, d)$ 即为问题的最优解。

2. 算法分析

- **时间复杂度**：

算法需要填满一个三维维度的状态表，其中第一维为物品数量 n ，第二维为载重容量 c ，第三维为容积容量 d 。每个状态的计算仅需 $O(1)$ 的时间。因此，总的时间复杂度为 $O(ncd)$ 。

- **空间复杂度**：

在直接实现中，需要存储三维数组，空间复杂度为 $O(ncd)$ 。若采用空间优化技术（如滚动数组），可以将空间复杂度进一步压缩至 $O(cd)$ 。

算法实现题3

3-3 石子合并问题。

问题描述：在一个圆形操场的四周摆放着 n 堆石子。现要将石子有次序地合并成一堆。规定每次只能选相邻的 2 堆石子合并成新的一堆,并将新的一堆石子数记为该次合并的得分。试设计一个算法,计算出将 n 堆石子合并成一堆的最小得分和最大得分。

算法设计：对于给定 n 堆石子, 计算合并成一堆的最小得分和最大得分。

数据输入：由文件 input.txt 提供输入数据。文件的第 1 行是正整数 n ($1 \leq n \leq 100$), 表示有 n 堆石子。第 2 行有 n 个数, 分别表示每堆石子的个数。

结果输出：将计算结果输出到文件 output.txt。文件第 1 行的数是最大得分, 第 2 行中的数是最大得分。

输入文件示例

input.txt

4

4 4 5 9

输出文件示例

output.txt

43

54

7

3 8

8 1 0

2 7 4 4

第3-3题答案:

1. 算法设计思想

本题是典型的**区间动态规划**问题。由于石子堆放在圆形操场四周, 属于环形结构, 首先需要通过“断环成链”的技巧将其转化为线性结构处理。

- **环形处理：**将原始长度为 n 的序列复制一份接在末尾, 形成长度为 $2n - 1$ 的线性序列。这样, 环中任意长度为 n 的连续子序列都可以通过在该长序列中取长度为 n 的区间来表示。
- **状态定义：**
 - 定义 $min_dp[i][j]$ 为合并第 i 堆到第 j 堆石子的最小得分。
 - 定义 $max_dp[i][j]$ 为合并第 i 堆到第 j 堆石子的最大得分。
 - 定义 $sum[i][j]$ 为第 i 堆到第 j 堆石子的总数量。

• 最优子结构与状态转移:

合并区间 $[i, j]$ 的石子可以看作是分别合并两个子区间 $[i, k]$ 和 $[k + 1, j]$ (其中 $i \leq k < j$), 最后再加上本次合并产生的得分 (即该区间内石子的总和)。

▪ 最小得分转移方程:

$$min_dp[i][j] = \min_{i \leq k < j} \{min_dp[i][k] + min_dp[k + 1][j]\} + \sum_{t=i}^j a[t]$$

▪ 最大得分转移方程:

$$max_dp[i][j] = \max_{i \leq k < j} \{max_dp[i][k] + max_dp[k + 1][j]\} + \sum_{t=i}^j a[t]$$

- **边界条件：**当 $i = j$ 时, 只有一堆石子, 不需要合并, 得分 $dp[i][i] = 0$ 。
- **求解目标：**

在 $2n - 1$ 的链上计算完所有长度为 n 的区间后, 最终答案为:

- 最小得分: $\min_{1 \leq i \leq n} \{min_dp[i][i + n - 1]\}$
- 最大得分: $\max_{1 \leq i \leq n} \{max_dp[i][i + n - 1]\}$

2. 算法分析

• 时间复杂度:

算法主要包含三层循环:

1. 第一层循环控制区间长度 len (从 2 到 n);
2. 第二层循环控制区间的起点 i ;
3. 第三层循环枚举区间分割点 k 。

对于扩充后的序列长度 $2n$, 计算次数约为 $(2n)^3$, 因此时间复杂度为 $O(n^3)$ 。

• 空间复杂度:

由于需要存储二维 DP 矩阵以及前缀和数组 (或区间和矩阵), 空间复杂度为 $O(n^2)$ 。

3-13

3-13 最大 k 乘积问题。

问题描述: 设 I 是一个 n 位十进制整数。如果将 I 划分为 k 段, 则可得到 k 个整数。这 k 个整数的乘积称为 I 的一个 k 乘积。试设计一个算法, 对于给定的 I 和 k , 求出 I 的最大 k 乘积。

算法设计: 对于给定的 I 和 k , 计算 I 的最大 k 乘积。

数据输入: 由文件 input.txt 提供输入数据。文件的第 1 行中有 2 个正整数 n 和 k 。正整数 n 是序列的长度, 正整数 k 是分割的段数。接下来的一行中是一个 n 位十进制整数 ($n \leq 10$)。

结果输出: 将计算结果输出到文件 output.txt。文件第 1 行中的数是计算出的最大 k 乘积。

输入文件示例

input.txt

2 1

15

输出文件示例

output.txt

15

第3-13题答案:

1. 算法设计思想

本题的目标是将一个 n 位十进制整数划分为 k 段, 使得各段整数的乘积最大。这是一个典型的**动态规划**问题, 通过将大整数的划分分解为规模较小的子问题来求解。

- **辅助定义:** 预处理并计算出整数 I 中从第 s 位开始、长度为 t 的连续数字所组成的十进制数值, 记为 $I(s, t)$ 。
- **状态定义:** 定义 $f(i, j)$ 表示将整数 I 的前 i 位数字划分为 j 段所能得到的最大乘积。
- **最优子结构与状态转移:**

为了求得 $f(i, j)$, 可以考虑最后一段 (即第 j 段) 的起始位置。假设前 $j - 1$ 段占据了前 m 位数字 (其中

$j-1 \leq m < i$), 那么最后一段即为从第 $m+1$ 位到第 i 位的数字。此时的乘积为前 $j-1$ 段的最大乘积 $f(m, j-1)$ 与最后一段数值的乘积。

通过枚举所有可能的分割点 m , 可以得到状态转移方程:

$$f(i, j) = \max_{j-1 \leq m < i} \{f(m, j-1) \times I(m+1, i-m)\}$$

- **边界条件:**

当 $j=1$ 时, 即只划分为一段, 最大乘积即为前 i 位数字组成的完整数值:

$$f(i, 1) = I(1, i), \quad 1 \leq i \leq n$$

- **求解目标:**

计算并填完状态表后, 最终 $f(n, k)$ 即为所求的 n 位整数划分为 k 段的最大乘积。

2. 算法分析

- **时间复杂度:**

1. **预处理阶段:** 计算所有可能的 $I(s, t)$ 需要 $O(n^2)$ 的时间。

2. **动态规划阶段:** 状态表的大小为 $n \times k$ 。对于每一个状态 $f(i, j)$, 需要枚举约 i 次分割点。因此, 填表的时间复杂度为 $O(n^2k)$ 。

综上所述, 总时间复杂度为 $O(n^2k)$ 。

- **空间复杂度:**

算法需要一个二维数组来存储 $f(i, j)$ 的状态值, 空间复杂度为 $O(nk)$ 。考虑到 $n \leq 10$ 的限制, 该算法在时空性能上非常高效。

3-14 最少费用购物问题。

问题描述：商店中每种商品都有标价。例如，一朵花的价格是 2 元，一个花瓶的价格是 5 元。为了吸引顾客，商店提供了一组优惠商品价。优惠商品是把一种或多种商品分成一组，并降价销售。例如，3 朵花的价格不是 6 元而是 5 元，2 个花瓶加 1 朵花的优惠价是 10 元。试设计一个算法，计算出某顾客所购商品应付的最少费用。

087

算法设计：对于给定欲购商品的价格和数量，以及优惠商品价，计算所购商品应付的最少费用。

数据输入：由文件 input.txt 提供欲购商品数据。文件的第 1 行中有 1 个整数 $B(0 \leq B \leq 5)$ ，表示所购商品种类数。在接下来的 B 行中，每行有 3 个数 C 、 K 和 P 。 C 表示商品的编码（每种商品有唯一编码）， $1 \leq C \leq 999$ ； K 表示购买该种商品总数， $1 \leq K \leq 5$ ； P 是该种商品的正常单价（每件商品的价格）， $1 \leq P \leq 999$ 。注意，一次最多可购买 $5 \times 5 = 25$ 件商品。

由文件 offer.txt 提供优惠商品价数据。文件的第 1 行中有 1 个整数 $S(0 \leq S \leq 99)$ ，表示共有 S 种优惠商品组合。接下来的 S 行，每行的第 1 个数描述优惠商品组合中商品的种类数 j 。接着是 j 个数字对 (C, K) ，其中 C 是商品编码， $1 \leq C \leq 999$ ； K 表示该种商品在此组合中的数量， $1 \leq K \leq 5$ 。每行最后一个数字 $P(1 \leq P \leq 9999)$ 表示此商品组合的优惠价。

结果输出：将计算出的所购商品应付的最少费用输出到文件 output.txt。

输入文件示例		输出文件示例
input.txt	offer.txt	output.txt
2	2	14
7 3 2	1 7 3 5	
8 2 5	2 7 1 8 2 10	

第3-14题答案:

1. 算法设计思想

本题的目标是在满足多种商品购买数量需求的前提下，通过合理组合单品购买与优惠礼包，使得总费用最少。该问题可以抽象为**多维 0-1 背包问题**，由于商品种类较少（不超过 5 种），且每种商品购买数量有限（不超过 5 件），可以使用**动态规划**解决。

• 状态定义：

由于最多涉及 5 种商品，定义五维状态数组 $cost(a, b, c, d, e)$ ，表示分别购买第 1 至第 5 种商品数量为 a, b, c, d, e 时所需的最小费用。

• 最优子结构与状态转移：

对于任意一种商品组合 (a, b, c, d, e) ，其最小费用取决于以下两种情况的最小值：

1. **单品购买方案**：即在 $cost(a-1, b, c, d, e)$ 的基础上，以正常单价购买一件第 1 种商品；其他商品同理。这实际上也可以看作是一种特殊的“优惠方案”。
2. **优惠礼包方案**：假设存在第 m 种优惠方案，其包含各类商品的数量分别为 $A[m], B[m], C[m], D[m], E[m]$ ，优惠价为 $offer(m)$ 。如果当前需求量满足该方案的消耗（即 $a \geq A[m], b \geq B[m] \dots$ ），则可以选择使用该方案。

状态转移方程：

$$cost(a, b, c, d, e) = \min\{\text{不使用优惠方案时的基础总价}, \min_{m=1}^S \{cost(a - A[m], b - B[m], \dots) + offer(m)\}\}$$

- **初始化：**

$cost(0, 0, 0, 0, 0) = 0$ ，其余状态初始化为无穷大。

- **求解目标：**

设顾客实际需要的 5 种商品数量分别为 p_1, p_2, p_3, p_4, p_5 ，最终计算出的 $cost(p_1, p_2, p_3, p_4, p_5)$ 即为应付的最少费用。

2. 算法分析

- **时间复杂度：**

状态空间的大小为 $6^5 = 7776$ （因为每种商品数量为 0-5 共 6 种可能）。对于每个状态，需要遍历所有的优惠方案 S （最多 99 种）。因此总的时间复杂度约为 $O(6^5 \times S)$ 。在给定的约束条件下，计算量非常小，能够快速得出结果。

- **空间复杂度：**

算法需要一个五维数组来存储所有可能购买组合的最小费用，空间复杂度为 $O(6^B)$ ，其中 B 是商品种类数（ $B \leq 5$ ）。这在内存消耗上是完全可以接受的。

3-17 字符串比较问题。

问题描述：对于长度相同的两个字符串 A 和 B ，其距离定义为相应位置字符距离之和。两个非空格字符的距离是它们的 ASCII 编码之差的绝对值。空格与空格的距离为 0，空格与其他字符的距离为一定值 k 。

在一般情况下，字符串 A 和 B 的长度不一定相同。字符串 A 的扩展是在 A 中插入若干空格字符所产生的字符串。在字符串 A 和 B 的所有长度相同的扩展中，有一对距离最小的扩展，该距离称为字符串 A 和 B 的扩展距离。

对于给定的字符串 A 和 B ，试设计一个算法，计算其扩展距离。

算法设计：对于给定的字符串 A 和 B ，计算其扩展距离。

数据输入：由文件 input.txt 给出输入数据。第 1 行是字符串 A ，第 2 行是字符串 B ，第 3 行是空格与其他字符的距离定值 k 。

结果输出：将计算出的字符串 A 和 B 的扩展距离输出到文件 output.txt。

输入文件示例

input.txt

cmc

snmn

2

输出文件示例

output.txt

10

第3-17题答案：**1. 算法设计思想**

本题的目标是求两个字符串 A 和 B 的扩展距离。通过在字符串中插入空格，使得扩展后的字符串长度相同，并最小化对应位置字符间的距离之和。该问题具有典型的**最优子结构**性质，可以通过**动态规划**求解。

• 距离定义细节：

- 两个非空格字符 a_i 和 b_j 的距离为 $|ASCII(a_i) - ASCII(b_j)|$ 。
- 空格与空格的距离为 0。
- 空格与其他字符的距离为给定常数 k 。

• 状态定义：

定义 $val[i][j]$ 表示字符串 A 的前 i 个字符与字符串 B 的前 j 个字符之间的最小扩展距离。

• 状态转移分析：

对于 $val[i][j]$ 的计算，考虑最后一步的匹配决策：

1. **字符 $A[i]$ 匹配字符 $B[j]$ ：**此时距离为 $val[i-1][j-1] + |ASCII(A[i]) - ASCII(B[j])|$ 。
2. **字符 $A[i]$ 匹配空格：**在字符串 B 中插入空格。此时距离为 $val[i-1][j] + k$ 。
3. **空格匹配字符 $B[j]$ ：**在字符串 A 中插入空格。此时距离为 $val[i][j-1] + k$ 。

注：由于算法的目标是最小化距离，且空格与字符距离为 $k > 0$ ，而空格与空格距离为 0，逻辑上不会

出现两个新插入空格相互匹配的情况（因为这不会使距离变小，反而增加了扩展长度），因此上述三种情况已涵盖所有最优可能的决策。

- **状态转移方程：**

$$val[i][j] = \min\{val[i-1][j-1] + dist(A[i], B[j]), val[i-1][j] + k, val[i][j-1] + k\}$$

- **边界条件：**

- $val[0][0] = 0$ （两个空串距离为 0）。
- $val[i][0] = i \times k$ （ A 的前 i 个字符全部与插入在 B 中的空格匹配）。
- $val[0][j] = j \times k$ （ B 的前 j 个字符全部与插入在 A 中的空格匹配）。

2. 算法分析

- **时间复杂度：**

算法需要填满一个 $(m+1) \times (n+1)$ 的二维状态表。每个状态的转移仅涉及常数次的比较与加法运算。因此，总时间复杂度为 $O(mn)$ ，其中 m, n 分别为字符串 A, B 的长度。

- **空间复杂度：**

算法使用了一个二维数组来记录子问题的解，空间复杂度为 $O(mn)$ 。